

QUESTION 12(a)

Question:

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("==== Single Pipe ====\\n");
    printf("Connecting: ls -l | wc -l\\n");

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        /* First command: ls -l */

        close(pipefd[0]);

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);
```

```

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    /* Second command: wc -l */

    close(pipefd[1]);

    /* Redirect stdin from pipe */
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("execlp wc");
    exit(1);
}

/* Parent closes both pipe ends */
close(pipefd[0]);
close(pipefd[1]);

/* Wait for both children */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Pipeline execution completed.\n");

return 0;

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skill12a.c
sri_dhruthi@Dhruthi:~$ gcc skill12a.c -o skill12a
sri_dhruthi@Dhruthi:~$ ./skill12a
===== Single Pipe =====
Connecting: ls -l | wc -l
110
Pipeline execution completed.

```

QUESTION 12(b)

Question:

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

This program demonstrates a **three-command pipeline**:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    pid_t pid1, pid2, pid3;

    /* Create first pipe */
    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        return 1;
    }

    /* Create second pipe */
    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        return 1;
    }

    printf("==== Multiple Pipe Pipeline ====\n");
    printf("Pipeline:\n");
    printf("ls -l | grep \"^-\" | wc -l\n\n");

    /* First process */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }
}
```

```

/* Second process */
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    /* grep "^-"
       stdin <- pipe1
       stdout -> pipe2
    */

    close(pipe1[1]);
    close(pipe2[0]);

    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);

    close(pipe1[0]);
    close(pipe2[1]);

    execlp("grep", "grep", "^-", NULL);

    perror("execlp grep");
    exit(1);
}

/* Third process */
pid3 = fork();

if (pid3 < 0)
{

```

```

if (pid3 == 0)
{
    /* wc -l */

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    /* stdin <- pipe2 */
    dup2(pipe2[0], STDIN_FILENO);

    close(pipe2[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("execlp wc");
    exit(1);
}

/* Parent closes all pipe descriptors */
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

/* Wait for all processes */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
waitpid(pid3, NULL, 0);

printf("All pipeline processes completed.\n");

return 0;
}

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skill12b.c
sri_dhruthi@Dhruthi:~$ gcc skill12b.c -o skill12b
sri_dhruthi@Dhruthi:~$ ./skill12b
==== Multiple Pipe Pipeline ====
Pipeline:
ls -l | grep "^-" | wc -l

110
All pipeline processes completed.

```